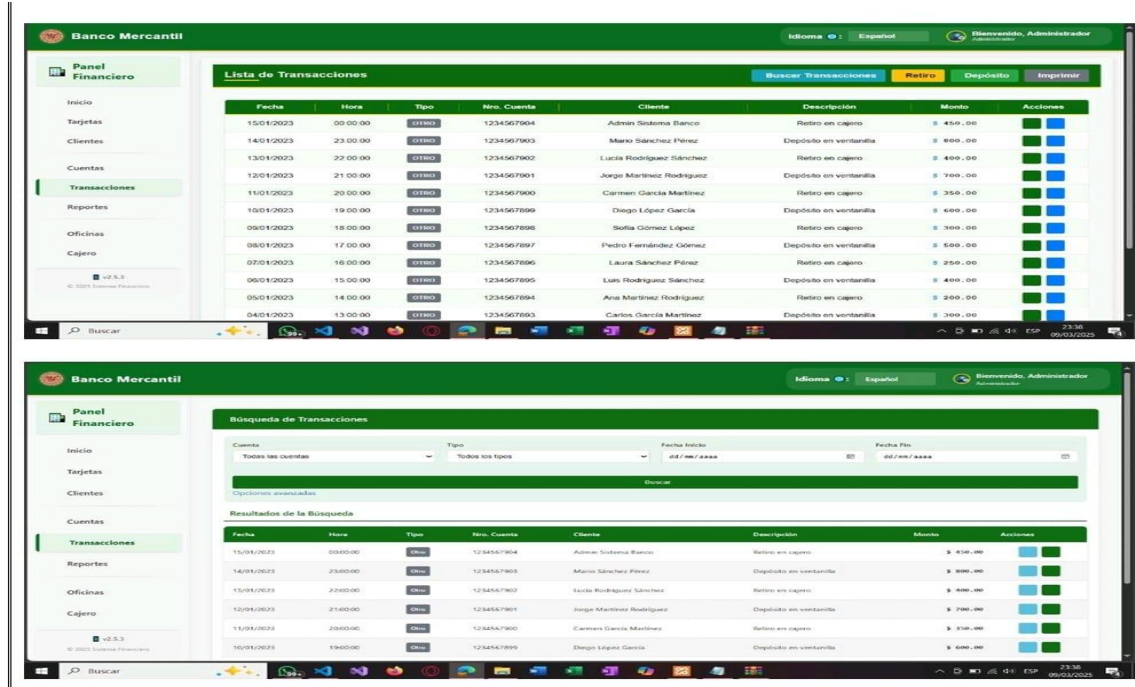
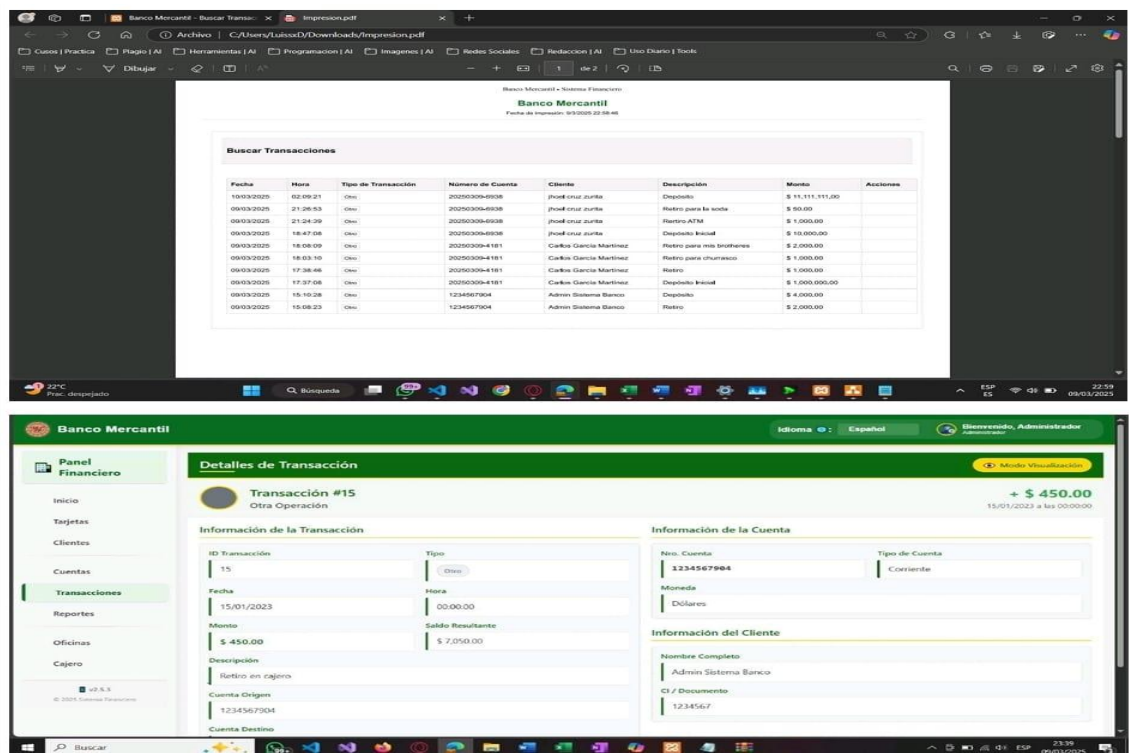


ACTIVIDAD 3

Modulo transacción



Impresión final



Código modulo transacción

```
<?php
```

```
class TransaccionController {
    // Propiedades
    private $db;
    private $session;
    private $model;

    // Constructor
    public function __construct() {
        // Inicializar conexión a la base de datos
        $database = new Database();
        $this->db = $database->connect();

        // Inicializar sesión
        $this->session = new Session();

        // Inicializar modelo
        $this->model = new Transaccion($this->db);
    }

    // Método para listar todas las transacciones
    public function listar() {
        global $lang;

        // Verificar si el usuario ha iniciado sesión
        if (!isset($_SESSION['user_id'])) {
            header('Location: index.php?controller=usuario&action=login');
            exit;
        }

        // Obtener todas las transacciones
        $resultado = $this->model->obtenerTodas();
        $transacciones = $resultado->fetchAll(PDO::FETCH_ASSOC);

        // Definir el título de la página
        $pageTitle = $lang['transaction_list'];

        // Determinar el controlador actual para marcar el menú
        $controller = 'transaccion';
    }
}
```

```

// Incluir la vista
$contentView = 'views/transacciones/listar.php';
include_once 'views/main.php';
}

// Método para mostrar el formulario de depósito
public function depositar() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Verificar si se proporcionó un ID de cuenta
    if (isset($_GET['idCuenta']) && !empty($_GET['idCuenta'])) {
        $idCuenta = (int)$_GET['idCuenta'];

        // Obtener datos de la cuenta
        $cuenta = new Cuenta($this->db);
        $cuenta->idCuenta = $idCuenta;

        if (!$cuenta->obtenerUna()) {
            $this->session->setFlashMessage('error', $lang['account_not_found']);
            header('Location: index.php?controller=cuenta&action=listar');
            exit;
        }

        // Verificar si la cuenta está activa
        // if ($cuenta->estado != 1) {
        //     $this->session->setFlashMessage('error', $lang['inactive_account_error']);
        //     header('Location: index.php?controller=cuenta&action=ver&id=' . $idCuenta);
        //     exit;
        // }

        // Obtener datos del cliente
        $cliente = new Cliente($this->db);
        $cliente->idPersona = $cuenta->idPersona;
        $cliente->obtenerUno();
    }

    // Definir título de la página
    $pageTitle = $lang['deposit_funds'];

    // Determinar el controlador actual para marcar el menú
    $controller = 'transaccion';

    // Definir vista a incluir
    $contentView = 'views/transacciones/depositar.php';

    // Mostrar plantilla principal

```

```

    include_once 'views/main.php';
}

// Método para buscar cuenta para depósito
public function buscarCuentaDeposito() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['nroCuenta']) ||
    empty($_POST['nroCuenta'])) {
        $this->session->setFlashMessage('error', $lang['account_number_required']);
        header('Location: index.php?controller=transaccion&action=depositar');
        exit;
    }

    $nroCuenta = trim($_POST['nroCuenta']);

    // Buscar la cuenta por número
    $cuentaModel = new Cuenta($this->db);
    $resultado = $cuentaModel->obtenerPorNumeroCuenta($nroCuenta);

    if ($resultado) {
        $idCuenta = $cuentaModel->idCuenta;

        // Redirigir al formulario de depósito con el ID de la cuenta
        header('Location: index.php?controller=transaccion&action=depositar&idCuenta=' .
        $idCuenta);
        exit;
    } else {
        $this->session->setFlashMessage('error', $lang['account_not_found']);
        header('Location: index.php?controller=transaccion&action=depositar');
        exit;
    }
}

// Método para procesar el depósito
public function procesarDeposito() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['idCuenta']) ||
    empty($_POST['idCuenta']) || !isset($_POST['monto']) || empty($_POST['monto'])) {

```

```

        $this->session->setFlashMessage('error', $lang['missing_required_fields']);
        header('Location: index.php?controller=transaccion&action=depositar');
        exit;
    }

    $idCuenta = (int)$_POST['idCuenta'];
    $monto = (float)$_POST['monto'];
    $descripcion = isset($_POST['descripcion']) ? trim($_POST['descripcion']) : $lang['deposit'];

    // Validar monto
    if ($monto <= 0) {
        $this->session->setFlashMessage('error', $lang['invalid_amount']);
        header('Location: index.php?controller=transaccion&action=depositar&idCuenta=' .
$idCuenta);
        exit;
    }

    try {
        // Realizar el depósito usando el modelo actualizado
        $this->model->realizarDeposito($idCuenta, $monto, $descripcion);

        $this->session->setFlashMessage('success', $lang['deposit_success']);
        header('Location: index.php?controller=cuenta&action=ver&id=' . $idCuenta);
        exit;
    } catch (Exception $e) {
        $this->session->setFlashMessage('error', $e->getMessage());
        header('Location: index.php?controller=transaccion&action=depositar&idCuenta=' .
$idCuenta);
        exit;
    }
}

// Método para mostrar el formulario de retiro
public function retirar() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Verificar si se proporcionó un ID de cuenta
    if (isset($_GET['idCuenta']) && !empty($_GET['idCuenta'])) {
        $idCuenta = (int)$_GET['idCuenta'];

        // Obtener datos de la cuenta
        $cuenta = new Cuenta($this->db);
        $cuenta->idCuenta = $idCuenta;

        if (!$cuenta->obtenerUna()) {
            $this->session->setFlashMessage('error', $lang['account_not_found']);

```

```

        header('Location: index.php?controller=cuenta&action=listar');
        exit;
    }

    // Verificar si la cuenta está activa
    //if ($cuenta->estado != 1) {
        // $this->session->setFlashMessage('error', $lang['inactive_account_error']);
        // header('Location: index.php?controller=transaccion&action=depositar&idCuenta=' .
        $idCuenta);
        // exit;
        //}

    // Obtener datos del cliente
    $cliente = new Cliente($this->db);
    $cliente->idPersona = $cuenta->idPersona;
    $cliente->obtenerUno();
}

// Definir título de la página
$pageTitle = $lang['withdraw_funds'];

// Determinar el controlador actual para marcar el menú
$controller = 'transaccion';

// Definir vista a incluir
$contentView = 'views/transacciones/retirar.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

// Método para buscar cuenta para retiro
public function buscarCuentaRetiro() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['nroCuenta']) ||
    empty($_POST['nroCuenta'])) {
        $this->session->setFlashMessage('error', $lang['account_number_required']);
        header('Location: index.php?controller=transaccion&action=retirar');
        exit;
    }

    $nroCuenta = trim($_POST['nroCuenta']);

    // Buscar la cuenta por número
    $cuentaModel = new Cuenta($this->db);

```

```

$resultado = $cuentaModel->obtenerPorNumeroCuenta($nroCuenta);

if ($resultado) {
    $idCuenta = $cuentaModel->idCuenta;

    // Redirigir al formulario de retiro con el ID de la cuenta
    header('Location: index.php?controller=transaccion&action=retirar&idCuenta=' .
$idCuenta);
    exit;
} else {
    $this->session->setFlashMessage('error', $lang['account_not_found']);
    header('Location: index.php?controller=transaccion&action=retirar');
    exit;
}
}

// Método para procesar el retiro
public function procesarRetiro() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['idCuenta']) ||
empty($_POST['idCuenta']) || !isset($_POST['monto']) || empty($_POST['monto'])) {
        $this->session->setFlashMessage('error', $lang['missing_required_fields']);
        header('Location: index.php?controller=transaccion&action=retirar');
        exit;
    }

    $idCuenta = (int)$_POST['idCuenta'];
    $monto = (float)$_POST['monto'];
    $descripcion = isset($_POST['descripcion']) ? trim($_POST['descripcion']) :
$lang['withdrawal'];

    // Validar monto
    if ($monto <= 0) {
        $this->session->setFlashMessage('error', $lang['invalid_amount']);
        header('Location: index.php?controller=transaccion&action=retirar&idCuenta=' .
$idCuenta);
        exit;
    }

    try {
        // Realizar el retiro usando el modelo actualizado
        $this->model->realizarRetiro($idCuenta, $monto, $descripcion);

        $this->session->setFlashMessage('success', $lang['withdrawal_success']);
        header('Location: index.php?controller=cuenta&action=ver&id=' . $idCuenta);
    }
}

```

```

        exit;
    } catch (Exception $e) {
        $this->session->setFlashMessage('error', $e->getMessage());
        header('Location: index.php?controller=transaccion&action=retirar&idCuenta=' .
$idCuenta);
        exit;
    }
}

// Método para mostrar el formulario de transferencia
public function transferir() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Variables para almacenar datos de cuenta de origen y destino
    $cuentaOrigen = true;
    $cuentaDestino = true;

    // Verificar si se proporcionó un ID de cuenta de origen
    if (isset($_GET['idCuentaOrigen']) && !empty($_GET['idCuentaOrigen'])) {
        $idCuentaOrigen = (int)$_GET['idCuentaOrigen'];

        // Obtener datos de la cuenta de origen
        $cuentaModel = new Cuenta($this->db);
        $cuentaModel->idCuenta = $idCuentaOrigen;

        if ($cuentaModel->obtenerUna()) {
            // Verificar si la cuenta está activa
            if ($cuentaModel->estado != 1) {
                $this->session->setFlashMessage('error', $lang['inactive_account_error']);
                header('Location: index.php?controller=cuenta&action=ver&id=' . $idCuentaOrigen);
                exit;
            }
        }

        $cuentaOrigen = [
            'idCuenta' => $cuentaModel->idCuenta,
            'nroCuenta' => $cuentaModel->nroCuenta,
            'tipoCuenta' => $cuentaModel->tipoCuenta,
            'tipoMoneda' => $cuentaModel->tipoMoneda,
            'saldo' => $cuentaModel->saldo,
            'estado' => $cuentaModel->estado
        ];

        // Obtener datos del cliente
        $clienteModel = new Cliente($this->db);
        $clienteModel->idPersona = $cuentaModel->idPersona;
        $clienteModel->obtenerUno();
    }
}

```



```

        // Añadir datos del cliente a la cuenta
        $cuentaOrigen['cliente_nombre'] = $clienteModel->nombre . ' ' . $clienteModel->apellidoPaterno . ' ' . $clienteModel->apellidoMaterno;
        $cuentaOrigen['cliente_ci'] = $clienteModel->ci;
        $cuentaOrigen['cliente_telefono'] = $clienteModel->telefono;
        $cuentaOrigen['cliente_email'] = $clienteModel->email;
    }
}

// Verificar si se proporcionó un ID de cuenta de destino
if (isset($_GET['idCuentaDestino']) && !empty($_GET['idCuentaDestino'])) {
    $idCuentaDestino = (int)$_GET['idCuentaDestino'];

    // Obtener datos de la cuenta de destino
    $cuentaModel = new Cuenta($this->db);
    $cuentaModel->idCuenta = $idCuentaDestino;

    if ($cuentaModel->obtenerUna()) {
        // Verificar si la cuenta está activa
        if ($cuentaModel->estado != 1) {
            $this->session->setFlashMessage('error', $lang['inactive_account_error']);
            header('Location:
index.php?controller=transaccion&action=transferir&idCuentaOrigen=' . $idCuentaOrigen);
            exit;
        }

        $cuentaDestino = [
            'idCuenta' => $cuentaModel->idCuenta,
            'nroCuenta' => $cuentaModel->nroCuenta,
            'tipoCuenta' => $cuentaModel->tipoCuenta,
            'tipoMoneda' => $cuentaModel->tipoMoneda,
            'estado' => $cuentaModel->estado
        ];

        // Obtener datos del cliente
        $clienteModel = new Cliente($this->db);
        $clienteModel->idPersona = $cuentaModel->idPersona;
        $clienteModel->obtenerUno();

        // Añadir datos del cliente a la cuenta
        $cuentaDestino['cliente_nombre'] = $clienteModel->nombre . ' ' . $clienteModel->apellidoPaterno . ' ' . $clienteModel->apellidoMaterno;
        $cuentaDestino['cliente_ci'] = $clienteModel->ci;
        $cuentaDestino['cliente_telefono'] = $clienteModel->telefono;
        $cuentaDestino['cliente_email'] = $clienteModel->email;
    }
}

// Definir título de la página
$pageTitle = $lang['transfer_funds'];

```

```

// Determinar el controlador actual para marcar el menú
$controller = 'transaccion';

// Definir vista a incluir
$contentView = 'views/transacciones/transferir.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

// Método para buscar cuenta de origen
public function buscarCuentaOrigen() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['nroCuentaOrigen']) ||
        empty($_POST['nroCuentaOrigen'])) {
        $this->session->setFlashMessage('error', $lang['account_number_required']);
        header('Location: index.php?controller=transaccion&action=transferir');
        exit;
    }

    $nroCuentaOrigen = trim($_POST['nroCuentaOrigen']);

    // Buscar la cuenta por número
    $cuentaModel = new Cuenta($this->db);
    $resultado = $cuentaModel->obtenerPorNumeroCuenta($nroCuentaOrigen);

    if ($resultado) {
        $idCuentaOrigen = $cuentaModel->idCuenta;

        // Redirigir al formulario de transferencia con el ID de la cuenta de origen
        header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
            $idCuentaOrigen);
        exit;
    } else {
        $this->session->setFlashMessage('error', $lang['account_not_found']);
        header('Location: index.php?controller=transaccion&action=transferir');
        exit;
    }
}

// Método para buscar cuenta de destino
public function buscarCuentaDestino() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión

```

```

if (!isset($_SESSION['user_id'])) {
    header('Location: index.php?controller=usuario&action=login');
    exit;
}

if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['idCuentaOrigen']) ||
empty($_POST['idCuentaOrigen']) || !isset($_POST['nroCuentaDestino']) ||
empty($_POST['nroCuentaDestino'])) {
    $this->session->setFlashMessage('error', $lang['missing_required_fields']);
    header('Location: index.php?controller=transaccion&action=transferir');
    exit;
}

$idCuentaOrigen = (int)$_POST['idCuentaOrigen'];
$nroCuentaDestino = trim($_POST['nroCuentaDestino']);

// Verificar que la cuenta de destino no sea la misma que la de origen
$cuentaOrigen = new Cuenta($this->db);
$cuentaOrigen->idCuenta = $idCuentaOrigen;
$cuentaOrigen->obtenerUna();

if ($cuentaOrigen->nroCuenta == $nroCuentaDestino) {
    $this->session->setFlashMessage('error', $lang['same_account_error']);
    header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
$idCuentaOrigen);
    exit;
}

// Buscar la cuenta de destino por número
$cuentaDestino = new Cuenta($this->db);
$resultado = $cuentaDestino->obtenerPorNumeroCuenta($nroCuentaDestino);

if ($resultado) {
    $idCuentaDestino = $cuentaDestino->idCuenta;

    // Redirigir al formulario de transferencia con ambos IDs de cuenta
    header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
$idCuentaOrigen . '&idCuentaDestino=' . $idCuentaDestino);
    exit;
} else {
    $this->session->setFlashMessage('error', $lang['destination_account_not_found']);
    header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
$idCuentaOrigen);
    exit;
}
}

// Método para procesar la transferencia
public function procesarTransferencia() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión

```

```

if (!isset($_SESSION['user_id'])) {
    header('Location: index.php?controller=usuario&action=login');
    exit;
}

if ($_SERVER['REQUEST_METHOD'] != 'POST' || !isset($_POST['idCuentaOrigen']) ||
empty($_POST['idCuentaOrigen']) || !isset($_POST['idCuentaDestino']) ||
empty($_POST['idCuentaDestino']) || !isset($_POST['monto']) || empty($_POST['monto'])) {
    $this->session->setFlashMessage('error', $lang['missing_required_fields']);
    header('Location: index.php?controller=transaccion&action=transferir');
    exit;
}

$idCuentaOrigen = (int)$_POST['idCuentaOrigen'];
$idCuentaDestino = (int)$_POST['idCuentaDestino'];
$monto = (float)$_POST['monto'];
$descripcion = isset($_POST['descripcion']) ? trim($_POST['descripcion']) : $lang['transfer'];

// Validar monto
if ($monto <= 0) {
    $this->session->setFlashMessage('error', $lang['invalid_amount']);
    header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
$idCuentaOrigen . '&idCuentaDestino=' . $idCuentaDestino);
    exit;
}

try {
    // Realizar la transferencia usando el modelo actualizado
    $this->model->realizarTransferencia($idCuentaOrigen, $idCuentaDestino, $monto,
$descripcion);

    $this->session->setFlashMessage('success', $lang['transfer_success']);
    header('Location: index.php?controller=cuenta&action=ver&id=' . $idCuentaOrigen);
    exit;
} catch (Exception $e) {
    $this->session->setFlashMessage('error', $e->getMessage());
    header('Location: index.php?controller=transaccion&action=transferir&idCuentaOrigen=' .
$idCuentaOrigen . '&idCuentaDestino=' . $idCuentaDestino);
    exit;
}
}

// Método para ver detalles de una transacción
public function ver() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }
}

```

```

// Verificar si se proporcionó un ID
if (!isset($_GET['id']) || empty($_GET['id'])) {
    $this->session->setFlashMessage('error', $lang['transaction_id_not_specified']);
    header('Location: index.php?controller=transaccion&action=listar');
    exit;
}

$id = (int)$_GET['id'];

// Obtener datos de la transacción
$this->model->idTransaccion = $id;
if (!$this->model->obtenerUna()) {
    $this->session->setFlashMessage('error', $lang['transaction_not_found']);
    header('Location: index.php?controller=transaccion&action=listar');
    exit;
}

// Obtener datos de la cuenta
$cuenta = new Cuenta($this->db);
$cuenta->idCuenta = $this->model->idCuenta;
$cuenta->obtenerUna();

// Obtener datos del cliente
$cliente = new Cliente($this->db);
$cliente->idPersona = $cuenta->idPersona;
$cliente->obtenerUno();

// Definir título de la página
$pageTitle = $lang['transaction_details'];

// Determinar el controlador actual para marcar el menú
$controller = 'transaccion';

// Definir vista a incluir
$contentView = 'views/transacciones/ver.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

// Método para buscar transacciones (filtrar por diferentes criterios)
public function buscar() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Inicializar variables de búsqueda

```

```

    $fechaInicio = isset($_GET['fechaInicio']) ? $_GET['fechaInicio'] : date('Y-m-01'); // Primer
    día del mes actual
    $fechaFin = isset($_GET['fechaFin']) ? $_GET['fechaFin'] : date('Y-m-d'); // Hoy
    $tipoTransaccion = isset($_GET['tipoTransaccion']) ? (int)$_GET['tipoTransaccion'] : 0; // 0 =
    Todas
    $idCuenta = isset($_GET['idCuenta']) ? (int)$_GET['idCuenta'] : 0; // 0 = Todas

    // Obtener transacciones filtradas
    $transacciones = [];

    if ($idCuenta > 0) {
        // Buscar por cuenta específica
        if ($fechaInicio && $fechaFin) {
            // Con rango de fechas
            $resultado = $this->model->obtenerPorRangoFechas($idCuenta, $fechaInicio,
            $fechaFin);
        } else {
            // Sin rango de fechas
            $resultado = $this->model->obtenerPorCuenta($idCuenta);
        }
    } else {
        // Buscar en todas las cuentas
        if ($fechaInicio && $fechaFin) {
            // Con rango de fechas
            $resultado = $this->model->obtenerPorRangoFechasGlobal($fechaInicio, $fechaFin);
        } else {
            // Sin rango de fechas, mostrar todas las transacciones
            $resultado = $this->model->obtenerTodas();
        }
    }

    // Filtrar por tipo de transacción si es necesario
    if ($resultado && $tipoTransaccion > 0) {
        $transaccionesTodas = $resultado->fetchAll(PDO::FETCH_ASSOC);
        foreach ($transaccionesTodas as $transaccion) {
            if ($transaccion['tipoTransaccion'] == $tipoTransaccion) {
                $transacciones[] = $transaccion;
            }
        }
    } else if ($resultado) {
        $transacciones = $resultado->fetchAll(PDO::FETCH_ASSOC);
    }

    // Obtener cuentas para el selector
    $cuentaModel = new Cuenta($this->db);
    $resultado = $cuentaModel->obtenerTodas();
    $cuentas = $resultado->fetchAll(PDO::FETCH_ASSOC);

    // Definir título de la página
    $pageTitle = $lang['search_transactions'];

    // Determinar el controlador actual para marcar el menú

```

```

$controller = 'transaccion';

// Definir vista a incluir
$contentView = 'views/transacciones/buscar.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

// Método para generar comprobante de transacción
public function comprobante() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Verificar si se proporcionó un ID
    if (!isset($_GET['id']) || empty($_GET['id'])) {
        $this->session->setFlashMessage('error', $lang['transaction_id_not_specified']);
        header('Location: index.php?controller=transaccion&action=listar');
        exit;
    }

    $id = (int)$_GET['id'];

    // Obtener datos de la transacción
    $this->model->idTransaccion = $id;
    if (!$this->model->obtenerUna()) {
        $this->session->setFlashMessage('error', $lang['transaction_not_found']);
        header('Location: index.php?controller=transaccion&action=listar');
        exit;
    }

    // Obtener datos de la cuenta
    $cuenta = new Cuenta($this->db);
    $cuenta->idCuenta = $this->model->idCuenta;
    $cuenta->obtenerUna();

    // Obtener datos del cliente
    $cliente = new Cliente($this->db);
    $cliente->idPersona = $cuenta->idPersona;
    $cliente->obtenerUno();

    // Definir título de la página
    $pageTitle = $lang['transaction_receipt'];

    // Determinar el controlador actual para marcar el menú
    $controller = 'transaccion';
    // Definir vista a incluir

```

```

$contentView = 'views/transacciones/comprobante.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

/**
 * Mostrar movimientos de una cuenta con saldo inicial y final
 * @return void
 */
public function movimientos() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Verificar si se proporcionó un ID de cuenta
    if (!isset($_GET['idCuenta']) || empty($_GET['idCuenta'])) {
        $this->session->setFlashMessage('error', $lang['account_id_not_specified']);
        header('Location: index.php?controller=cuenta&action=listar');
        exit;
    }

    $idCuenta = (int)$_GET['idCuenta'];

    // Obtener datos de la cuenta
    $cuenta = new Cuenta($this->db);
    $cuenta->idCuenta = $idCuenta;

    if (!$cuenta->obtenerUna()) {
        $this->session->setFlashMessage('error', $lang['account_not_found']);
        header('Location: index.php?controller=cuenta&action=listar');
        exit;
    }

    // Obtener datos del cliente
    $cliente = new Cliente($this->db);
    $cliente->idPersona = $cuenta->idPersona;
    $cliente->obtenerUno();

    // Obtener fechas para filtrar
    $fechaFin = isset($_GET['fechaFin']) ? $_GET['fechaFin'] : date('Y-m-d');
    $fechaInicio = isset($_GET['fechaInicio']) ? $_GET['fechaInicio'] : date('Y-m-01',
    strtotime($fechaFin)); // Primer día del mes

    // Calcular saldo inicial (saldo en la fecha de inicio)
    $saldoInicial = $this->calcularSaldoInicial($idCuenta, $fechaInicio);

    // Obtener movimientos en el rango de fechas

```



```

$resultado = $this->model->obtenerPorRangoFechas($idCuenta, $fechaInicio, $fechaFin);
$movimientos = $resultado->fetchAll(PDO::FETCH_ASSOC);

// Calcular totales
$totalIngresos = 0;
$totalEgresos = 0;

foreach ($movimientos as $movimiento) {
    if ($movimiento['tipoTransaccion'] == 2 || $movimiento['tipoTransaccion'] == 3) {
        // Depósitos y transferencias recibidas
        $totalIngresos += $movimiento['monto'];
    } else {
        // Retiros y transferencias enviadas
        $totalEgresos += $movimiento['monto'];
    }
}

// Calcular saldo final
$saldoFinal = $saldoInicial + $totalIngresos - $totalEgresos;

// Definir título de la página
$pageTitle = $lang['account_movements'];

// Determinar el controlador actual para marcar el menú
$controller = 'transaccion';

// Definir vista a incluir
$contentView = 'views/transacciones/movimientos.php';

// Mostrar plantilla principal
include_once 'views/main.php';
}

/**
 * Calcular saldo inicial de una cuenta en una fecha específica
 * @param int $idCuenta ID de la cuenta
 * @param string $fecha Fecha para calcular el saldo inicial (YYYY-MM-DD)
 * @return float Saldo inicial
 */
private function calcularSaldoInicial($idCuenta, $fecha) {
    // Obtener la cuenta
    $cuenta = new Cuenta($this->db);
    $cuenta->idCuenta = $idCuenta;
    $cuenta->obtenerUna();

    // Obtener todas las transacciones anteriores a la fecha
    $query = "SELECT
        SUM(CASE WHEN tipoTransaccion IN (2, 3) THEN monto ELSE 0 END) as
total_ingresos,
        SUM(CASE WHEN tipoTransaccion IN (1, 4) THEN monto ELSE 0 END) as
total_egresos
    FROM Transaccion

```

```

        WHERE idCuenta = :idCuenta AND fecha < :fecha";

$stmt = $this->db->prepare($query);
$stmt->bindParam(':idCuenta', $idCuenta);
$stmt->bindParam(':fecha', $fecha);
$stmt->execute();

$resultado = $stmt->fetch(PDO::FETCH_ASSOC);

// Calcular saldo inicial
$totalIngresos = $resultado['total_ingresos'] ?: 0;
$totalEgresos = $resultado['total_egresos'] ?: 0;

// Si la cuenta se abrió después de la fecha de inicio, el saldo inicial debe ser 0
if (strtotime($cuenta->fechaApertura) > strtotime($fecha)) {
    return 0;
}

// Calcular saldo inicial restando transacciones anteriores al saldo actual
$saldoInicial = $cuenta->saldo - $totalIngresos + $totalEgresos;

return $saldoInicial;
}

/**
 * Imprimir movimientos de cuenta
 * @return void
 */
public function imprimirMovimientos() {
    global $lang;

    // Verificar si el usuario ha iniciado sesión
    if (!isset($_SESSION['user_id'])) {
        header('Location: index.php?controller=usuario&action=login');
        exit;
    }

    // Verificar si se proporcionaron los parámetros necesarios
    if (!isset($_GET['idCuenta']) || empty($_GET['idCuenta'])) {
        echo $lang['account_id_not_specified'];
        exit;
    }

    $idCuenta = (int)$_GET['idCuenta'];
    $fechaFin = isset($_GET['fechaFin']) ? $_GET['fechaFin'] : date('Y-m-d');
    $fechaInicio = isset($_GET['fechaInicio']) ? $_GET['fechaInicio'] : date('Y-m-01',
        strtotime($fechaFin));

    // Obtener datos de la cuenta
    $cuenta = new Cuenta($this->db);
    $cuenta->idCuenta = $idCuenta;

```

```

if (!$cuenta->obtenerUna()) {
    echo $lang['account_not_found'];
    exit;
}

// Obtener datos del cliente
$ccliente = new Cliente($this->db);
$ccliente->idPersona = $cuenta->idPersona;
$ccliente->obtenerUno();

// Calcular saldo inicial
$saldoInicial = $this->calcularSaldoInicial($idCuenta, $fechaInicio);

// Obtener movimientos
$resultado = $this->model->obtenerPorRangoFechas($idCuenta, $fechaInicio, $fechaFin);
$movimientos = $resultado->fetchAll(PDO::FETCH_ASSOC);

// Calcular totales
$totalIngresos = 0;
$totalEgresos = 0;

foreach ($movimientos as $movimiento) {
    if ($movimiento['tipoTransaccion'] == 2 || $movimiento['tipoTransaccion'] == 3) {
        // Depósitos y transferencias recibidas
        $totalIngresos += $movimiento['monto'];
    } else {
        // Retiros y transferencias enviadas
        $totalEgresos += $movimiento['monto'];
    }
}

// Calcular saldo final
$saldoFinal = $saldoInicial + $totalIngresos - $totalEgresos;

// Mostrar la vista para impresión
include 'views/transacciones/imprimir_movimientos.php';
}
}

```

>

Model/transacción .php

```
<?php
/**
 * Modelo Transacción Mejorado
 * Gestiona las operaciones con transacciones en la base de datos
 * Fecha de actualización: 09/03/2025
 */
class Transaccion {
    // Conexión a la base de datos
    private $conn;

    // Propiedades de la transacción
    public $idTransaccion;
    public $tipoTransaccion; // 1: Retiro, 2: Depósito, 3: Transferencia recibida, 4:
    Transferencia enviada
    public $fecha;
    public $hora;
    public $descripcion;
    public $monto;
    public $idCuenta;
    public $saldoResultante;
    public $cuentaOrigen;
    public $cuentaDestino;
    public $hash;

    /**
     * Constructor con DB
     * @param PDO $db
     */
    public function __construct($db) {
        $this->conn = $db;
    }

    /**
     * Obtener todas las transacciones
     * @return PDOStatement
     */
    public function obtenerTodas() {
        $query = "SELECT t.*, c.nroCuenta,
            CONCAT(p.nombre, ' ', p.apellidoPaterno, ' ', p.apellidoMaterno) as
            cliente_nombre
            FROM Transaccion t
            INNER JOIN Cuenta c ON t.idCuenta = c.idCuenta
            INNER JOIN Persona p ON c.idPersona = p.idPersona
            ORDER BY t.fecha DESC, t.hora DESC";

        $stmt = $this->conn->prepare($query);
        $stmt->execute();

        return $stmt;
    }
}
```

```

}

/**
 * Obtener una transacción
 * @return boolean
 */
public function obtenerUna() {
    $query = "SELECT t.*, c.nroCuenta,
        CONCAT(p.nombre, ' ', p.apellidoPaterno, ' ', p.apellidoMaterno) as
cliente_nombre
        FROM Transaccion t
        INNER JOIN Cuenta c ON t.idCuenta = c.idCuenta
        INNER JOIN Persona p ON c.idPersona = p.idPersona
        WHERE t.idTransaccion = :id";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':id', $this->idTransaccion);
    $stmt->execute();

    $row = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($row) {
        $this->tipoTransaccion = $row['tipoTransaccion'];
        $this->fecha = $row['fecha'];
        $this->hora = $row['hora'];
        $this->descripcion = $row['descripcion'];
        $this->monto = $row['monto'];
        $this->idCuenta = $row['idCuenta'];
        $this->saldoResultante = isset($row['saldoResultante']) ? $row['saldoResultante'] :
true;
        $this->cuentaOrigen = isset($row['cuentaOrigen']) ? $row['cuentaOrigen'] : true;
        $this->cuentaDestino = isset($row['cuentaDestino']) ? $row['cuentaDestino'] : true;
        $this->hash = isset($row['hash']) ? $row['hash'] : true;
        return true;
    }

    return false;
}

/**
 * Obtener transacciones por cuenta
 * @param int $idCuenta
 * @return PDOStatement
 */
public function obtenerPorCuenta($idCuenta) {
    $query = "SELECT * FROM Transaccion
        WHERE idCuenta = :idCuenta
        ORDER BY fecha DESC, hora DESC";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':idCuenta', $idCuenta);
    $stmt->execute();

```

```

    return $stmt;
}

/**
 * Obtener transacciones por cuenta y rango de fechas
 * @param int $idCuenta
 * @param string $fechaInicio
 * @param string $fechaFin
 * @return PDOStatement
 */
public function obtenerPorRangoFechas($idCuenta, $fechaInicio, $fechaFin) {
    $query = "SELECT * FROM Transaccion
        WHERE idCuenta = :idCuenta
        AND fecha BETWEEN :fechaInicio AND :fechaFin
        ORDER BY fecha DESC, hora DESC";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':idCuenta', $idCuenta);
    $stmt->bindParam(':fechaInicio', $fechaInicio);
    $stmt->bindParam(':fechaFin', $fechaFin);
    $stmt->execute();

    return $stmt;
}

/**
 * Obtener transacciones por rango de fechas global (todos los usuarios)
 * @param string $fechaInicio
 * @param string $fechaFin
 * @return PDOStatement
 */
public function obtenerPorRangoFechasGlobal($fechaInicio, $fechaFin) {
    $query = "SELECT t.*, c.nroCuenta,
        CONCAT(p.nombre, ' ', p.apellidoPaterno, ' ', p.apellidoMaterno) as
cliente_nombre
        FROM Transaccion t
        INNER JOIN Cuenta c ON t.idCuenta = c.idCuenta
        INNER JOIN Persona p ON c.idPersona = p.idPersona
        WHERE t.fecha BETWEEN :fechaInicio AND :fechaFin
        ORDER BY t.fecha DESC, t.hora DESC";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':fechaInicio', $fechaInicio);
    $stmt->bindParam(':fechaFin', $fechaFin);
    $stmt->execute();

    return $stmt;
}

/**
 * Crear una transacción

```

```

* @return boolean
*/
public function crear() {
    try {
        // Si no se proporciona fecha y hora, usar la fecha y hora actual
        if (empty($this->fecha)) {
            $this->fecha = date('Y-m-d');
        }

        if (empty($this->hora)) {
            $this->hora = date('H:i:s');
        }

        // Crear la transacción
        $query = "INSERT INTO Transaccion
            (tipoTransaccion, fecha, hora, descripcion, monto, idCuenta,
            saldoResultante, cuentaOrigen, cuentaDestino, hash)
            VALUES
            (:tipoTransaccion, :fecha, :hora, :descripcion, :monto, :idCuenta,
            :saldoResultante, :cuentaOrigen, :cuentaDestino, :hash)";

        $stmt = $this->conn->prepare($query);

        // Limpiar datos
        $this->descripcion = htmlspecialchars(strip_tags($this->descripcion));

        // Vincular datos
        $stmt->bindParam(':tipoTransaccion', $this->tipoTransaccion);
        $stmt->bindParam(':fecha', $this->fecha);
        $stmt->bindParam(':hora', $this->hora);
        $stmt->bindParam(':descripcion', $this->descripcion);
        $stmt->bindParam(':monto', $this->monto);
        $stmt->bindParam(':idCuenta', $this->idCuenta);
        $stmt->bindParam(':saldoResultante', $this->saldoResultante);
        $stmt->bindParam(':cuentaOrigen', $this->cuentaOrigen);
        $stmt->bindParam(':cuentaDestino', $this->cuentaDestino);
        $stmt->bindParam(':hash', $this->hash);

        // Ejecutar consulta
        $stmt->execute();

        // Obtener el ID de la transacción creada
        $this->idTransaccion = $this->conn->lastInsertId();
        return true;
    } catch (Exception $e) {
        error_log("Error al crear transacción: " . $e->getMessage());
        return false;
    }
}

/**

```

```

* Crear transacción en una transacción de DB
* Para usarse cuando se manejan varias operaciones en una sola transacción
* @return boolean
*/
public function crearEnTransaccion() {
    try {
        // Si no se proporciona fecha y hora, usar la fecha y hora actual
        if (empty($this->fecha)) {
            $this->fecha = date('Y-m-d');
        }

        if (empty($this->hora)) {
            $this->hora = date('H:i:s');
        }

        // Crear la transacción
        $query = "INSERT INTO Transaccion
            (tipoTransaccion, fecha, hora, descripcion, monto, idCuenta,
            saldoResultante, cuentaOrigen, cuentaDestino, hash)
            VALUES
            (:tipoTransaccion, :fecha, :hora, :descripcion, :monto, :idCuenta,
            :saldoResultante, :cuentaOrigen, :cuentaDestino, :hash)";

        $stmt = $this->conn->prepare($query);

        // Limpiar datos
        $this->descripcion = htmlspecialchars(strip_tags($this->descripcion));

        // Vincular datos
        $stmt->bindParam(':tipoTransaccion', $this->tipoTransaccion);
        $stmt->bindParam(':fecha', $this->fecha);
        $stmt->bindParam(':hora', $this->hora);
        $stmt->bindParam(':descripcion', $this->descripcion);
        $stmt->bindParam(':monto', $this->monto);
        $stmt->bindParam(':idCuenta', $this->idCuenta);
        $stmt->bindParam(':saldoResultante', $this->saldoResultante);
        $stmt->bindParam(':cuentaOrigen', $this->cuentaOrigen);
        $stmt->bindParam(':cuentaDestino', $this->cuentaDestino);
        $stmt->bindParam(':hash', $this->hash);

        // Ejecutar consulta
        $stmt->execute();

        // Obtener el ID de la transacción creada
        $this->idTransaccion = $this->conn->lastInsertId();
        return true;
    } catch (Exception $e) {
        throw new Exception("Error al crear transacción: " . $e->getMessage());
    }
}

```



```

/**
 * Realiza un depósito en la cuenta
 * @param int $idCuenta ID de la cuenta
 * @param float $monto Monto a depositar
 * @param string $descripcion Descripción de la transacción
 * @return boolean
 */
public function realizarDeposito($idCuenta, $monto, $descripcion = 'Depósito') {
    // Iniciar transacción en la base de datos
    $this->conn->beginTransaction();

    try {
        // Obtener cuenta
        $cuenta = new Cuenta($this->conn);
        $cuenta->idCuenta = $idCuenta;

        if (!$cuenta->obtenerUna()) {
            throw new Exception("Cuenta no encontrada.");
        }

        // Verificar que la cuenta esté activa
        // if ($cuenta->estado = 1) {
        //     throw new Exception("La cuenta no está activa.");
        // }

        // Actualizar saldo de la cuenta
        $nuevoSaldo = $cuenta->saldo + $monto;

        $query = "UPDATE Cuenta SET saldo = :saldo WHERE idCuenta = :idCuenta";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':saldo', $nuevoSaldo);
        $stmt->bindParam(':idCuenta', $idCuenta);
        $stmt->execute();

        // Crear transacción
        $this->idCuenta = $idCuenta;
        $this->tipoTransaccion = 2; // Depósito
        $this->monto = $monto;
        $this->descripcion = $descripcion;
        $this->fecha = date('Y-m-d');
        $this->hora = date('H:i:s');
        $this->saldoResultante = $nuevoSaldo;
        $this->hash = bin2hex(random_bytes(16)); // Generar hash único

        if (!$this->crearEnTransaccion()) {
            throw new Exception("Error al registrar el depósito.");
        }

        // Confirmar transacción
        $this->conn->commit();
        return true;
    }
}

```

```

    } catch (Exception $e) {
        // Revertir transacción en caso de error
        $this->conn->rollBack();
        error_log("Error en realizarDeposito: " . $e->getMessage());
        throw $e;
    }
}

/**
 * Realiza un retiro de la cuenta
 * @param int $idCuenta ID de la cuenta
 * @param float $monto Monto a retirar
 * @param string $descripcion Descripción de la transacción
 * @return boolean
 */
public function realizarRetiro($idCuenta, $monto, $descripcion = 'Retiro') {
    // Iniciar transacción en la base de datos
    $this->conn->beginTransaction();

    try {
        // Obtener cuenta
        $cuenta = new Cuenta($this->conn);
        $cuenta->idCuenta = $idCuenta;

        if (!$cuenta->obtenerUna()) {
            throw new Exception("Cuenta no encontrada.");
        }

        // Verificar que la cuenta esté activa
        //if ($cuenta->estado != 1) {
        //    throw new Exception("La cuenta no está activa.");
        // }

        // Verificar saldo suficiente
        if ($cuenta->saldo < $monto) {
            throw new Exception("Saldo insuficiente.");
        }

        // Actualizar saldo de la cuenta
        $nuevoSaldo = $cuenta->saldo - $monto;

        $query = "UPDATE Cuenta SET saldo = :saldo WHERE idCuenta = :idCuenta";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':saldo', $nuevoSaldo);
        $stmt->bindParam(':idCuenta', $idCuenta);
        $stmt->execute();

        // Crear transacción
        $this->idCuenta = $idCuenta;
        $this->tipoTransaccion = 1; // Retiro
        $this->monto = $monto;
        $this->descripcion = $descripcion;
    }
}

```

```

        $this->fecha = date('Y-m-d');
        $this->hora = date('H:i:s');
        $this->saldoResultante = $nuevoSaldo;
        $this->hash = bin2hex(random_bytes(16)); // Generar hash único

        if (!$this->crearEnTransaccion()) {
            throw new Exception("Error al registrar el retiro.");
        }

        // Confirmar transacción
        $this->conn->commit();
        return true;

    } catch (Exception $e) {
        // Revertir transacción en caso de error
        $this->conn->rollBack();
        error_log("Error en realizarRetiro: " . $e->getMessage());
        throw $e;
    }
}

/**
 * Realiza una transferencia entre cuentas
 * @param int $idCuentaOrigen ID de la cuenta origen
 * @param int $idCuentaDestino ID de la cuenta destino
 * @param float $monto Monto a transferir
 * @param string $descripcion Descripción de la transacción
 * @return boolean
 */
public function realizarTransferencia($idCuentaOrigen, $idCuentaDestino, $monto,
$descripcion = 'Transferencia') {
    // Iniciar transacción en la base de datos
    $this->conn->beginTransaction();

    try {
        // Obtener cuenta origen
        $cuentaOrigen = new Cuenta($this->conn);
        $cuentaOrigen->idCuenta = $idCuentaOrigen;

        if (!$cuentaOrigen->obtenerUna()) {
            throw new Exception("Cuenta origen no encontrada.");
        }

        // Verificar que la cuenta origen esté activa
        if ($cuentaOrigen->estado != 1) {
            throw new Exception("La cuenta origen no está activa.");
        }

        // Verificar saldo suficiente
        if ($cuentaOrigen->saldo < $monto) {
            throw new Exception("Saldo insuficiente en la cuenta origen.");
        }
    }
}

```

```

// Obtener cuenta destino
$cuentaDestino = new Cuenta($this->conn);
$cuentaDestino->idCuenta = $idCuentaDestino;

if (!$cuentaDestino->obtenerUna()) {
    throw new Exception("Cuenta destino no encontrada.");
}

// Verificar que la cuenta destino esté activa
// if ($cuentaDestino->estado != 1) {
//     throw new Exception("La cuenta destino no está activa.");
// }

// Verificar que las monedas sean compatibles
if ($cuentaOrigen->tipoMoneda != $cuentaDestino->tipoMoneda) {
    throw new Exception("Las monedas de las cuentas no son compatibles.");
}

// Actualizar saldo de la cuenta origen (restar)
$nuevoSaldoOrigen = $cuentaOrigen->saldo - $monto;

$queryOrigen = "UPDATE Cuenta SET saldo = :saldo WHERE idCuenta = :idCuenta";
$stmtOrigen = $this->conn->prepare($queryOrigen);
$stmtOrigen->bindParam(':saldo', $nuevoSaldoOrigen);
$stmtOrigen->bindParam(':idCuenta', $idCuentaOrigen);
$stmtOrigen->execute();

// Actualizar saldo de la cuenta destino (sumar)
$nuevoSaldoDestino = $cuentaDestino->saldo + $monto;

$queryDestino = "UPDATE Cuenta SET saldo = :saldo WHERE idCuenta = :idCuenta";
$stmtDestino = $this->conn->prepare($queryDestino);
$stmtDestino->bindParam(':saldo', $nuevoSaldoDestino);
$stmtDestino->bindParam(':idCuenta', $idCuentaDestino);
$stmtDestino->execute();

// Generar hash único para ambas transacciones
$hashTransaccion = bin2hex(random_bytes(16));

// Crear transacción para la cuenta origen (salida de dinero)
$this->idCuenta = $idCuentaOrigen;
$this->tipoTransaccion = 4; // Transferencia enviada
$this->monto = $monto;
$this->descripcion = $descripcion;
$this->fecha = date('Y-m-d');
$this->hora = date('H:i:s');
$this->saldoResultante = $nuevoSaldoOrigen;
$this->cuentaDestino = $cuentaDestino->nroCuenta;
$this->hash = $hashTransaccion;

if (!$this->crearEnTransaccion()) {

```

```

        throw new Exception("Error al registrar la transferencia (origen).");
    }

    // Crear transacción para la cuenta destino (entrada de dinero)
    $transaccionDestino = new Transaccion($this->conn);
    $transaccionDestino->idCuenta = $idCuentaDestino;
    $transaccionDestino->tipoTransaccion = 3; // Transferencia recibida
    $transaccionDestino->monto = $monto;
    $transaccionDestino->descripcion = $descripcion;
    $transaccionDestino->fecha = date('Y-m-d');
    $transaccionDestino->hora = date('H:i:s');
    $transaccionDestino->saldoResultante = $nuevoSaldoDestino;
    $transaccionDestino->cuentaOrigen = $cuentaOrigen->nroCuenta;
    $transaccionDestino->hash = $hashTransaccion;

    if (!$transaccionDestino->crearEnTransaccion()) {
        throw new Exception("Error al registrar la transferencia (destino).");
    }

    // Confirmar transacción
    $this->conn->commit();
    return true;
} catch (Exception $e) {
    // Revertir transacción en caso de error
    $this->conn->rollBack();
    error_log("Error en realizarTransferencia: " . $e->getMessage());
    throw $e;
}
}

/**
 * Contar transacciones del día
 * @return int
 */
public function contarTransaccionesHoy() {
    $hoy = date('Y-m-d');
    $query = "SELECT COUNT(*) as total FROM Transaccion WHERE fecha = :fecha";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':fecha', $hoy);
    $stmt->execute();
    $row = $stmt->fetch(PDO::FETCH_ASSOC);

    return $row['total'];
}

/**
 * Obtener transacciones recientes
 * @param int $limit Número de transacciones a obtener
 * @return PDOStatement
 */

```

```

public function obtenerRecientes($limit = 10) {
    $query = "SELECT t.*, c.nroCuenta,
        CONCAT(p.nombre, ' ', p.apellidoPaterno, ' ', p.apellidoMaterno) as
cliente_nombre
        FROM Transaccion t
        INNER JOIN Cuenta c ON t.idCuenta = c.idCuenta
        INNER JOIN Persona p ON c.idPersona = p.idPersona
        ORDER BY t.fecha DESC, t.hora DESC
        LIMIT :limit";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':limit', $limit, PDO::PARAM_INT);
    $stmt->execute();

    return $stmt;
}
}

```

Vista/transacción.php

```

<!DOCTYPE html>

<html lang="es">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Comprobante de Transacción - Banco Mercantil</title>

    <link href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css"
rel="stylesheet">

    <style>

        /* Variables globales */

        :root {

            --primary: #0f6e14;

            --primary-light: #e9f5ea;

            --secondary: #f4c01e;

            --secondary-light: #fff8e1;

            --success: #5cb85c;

            --info: #5bc0de;

            --warning: #f0ad4e;

            --danger: #d9534f;

```

```
--text-dark: #333333;

--text-medium: #666666;

--text-light: #999999;

--gray-light: #f8f8f8;

--gray: #e9e9e9;

--border: #e0e0e0;

--border-radius: 8px;

--box-shadow: 0 2px 15px rgba(0, 0, 0, 0.1);

}
```

```
/* Estilos generales */
```

```
* {

  margin: 0;

  padding: 0;

  box-sizing: border-box;

}
```

```
html, body {

  height: 100%;

  width: 100%;

  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

  background-color: #f9f9f9;

  color: var(--text-dark);

  line-height: 1.6;

}
```

```
.container {

  width: 100%;

  height: 100%;

  padding: 20px;

  display: flex;
```

```
    flex-direction: column;
}
```

```
/* Estilos del comprobante */
```

```
.receipt-container {
    background: white;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    overflow: hidden;
    position: relative;
    flex: 1;
    display: flex;
    flex-direction: column;
}
```

```
/* Marca de agua */
```

```
.watermark {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%) rotate(-30deg);
    font-size: 120px;
    opacity: 0.03;
    font-weight: bold;
    color: var(--primary);
    z-index: 0;
    user-select: none;
    white-space: nowrap;
}
```

```
/* Encabezado del comprobante */
```



```
.receipt-header {  
  background-color: var(--primary);  
  color: white;  
  padding: 20px;  
  text-align: center;  
  position: relative;  
  border-bottom: 5px solid var(--secondary);  
}
```

```
.receipt-title {  
  font-size: 1.6rem;  
  font-weight: 600;  
  margin-bottom: 10px;  
  letter-spacing: 0.5px;  
  text-transform: uppercase;  
}
```

```
.receipt-subtitle {  
  font-size: 1.1rem;  
  font-weight: 500;  
  color: rgba(255, 255, 255, 0.9);  
  margin-top: 10px;  
}
```

```
.bank-logo {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  margin-bottom: 15px;  
}
```

```
.bank-logo img {  
  width: 60px;  
  height: 60px;  
  border-radius: 50%;  
  background-color: white;  
  padding: 5px;  
  margin-right: 15px;  
}
```

```
.bank-name {  
  font-size: 1.8rem;  
  font-weight: 700;  
}
```

/ Información de transacción en encabezado */*

```
.header-transaction-info {  
  background-color: rgba(255, 255, 255, 0.1);  
  border-radius: var(--border-radius);  
  padding: 10px 15px;  
  margin: 15px 0 5px;  
}
```

```
.header-transaction-row {  
  display: flex;  
  justify-content: space-between;  
  flex-wrap: wrap;  
}
```

```
.header-transaction-item {  
  padding: 5px 10px;  
  font-size: 0.9rem;
```

```
}
```

```
.header-label {  
  font-weight: 600;  
  color: rgba(255, 255, 255, 0.9);  
}
```

```
.header-value {  
  color: white;  
}
```

```
/* Cuerpo del comprobante */
```

```
.receipt-body {  
  padding: 30px;  
  position: relative;  
  z-index: 1;  
  flex: 1;  
  overflow-y: auto;  
}
```

```
.transaction-info {  
  background-color: var(--gray-light);  
  border-radius: var(--border-radius);  
  padding: 20px;  
  margin-bottom: 25px;  
  border-left: 4px solid var(--primary);  
}
```

```
.transaction-type {  
  display: inline-block;  
  padding: 8px 15px;
```

```
background-color: var(--primary);  
color: white;  
border-radius: 30px;  
font-weight: 600;  
font-size: 0.9rem;  
margin-bottom: 10px;  
}
```

```
.transaction-type.withdrawal {  
  background-color: var(--warning);  
}
```

```
.transaction-type.deposit {  
  background-color: var(--success);  
}
```

```
.transaction-type.transfer-in {  
  background-color: var(--info);  
}
```

```
.transaction-type.transfer-out {  
  background-color: var(--primary);  
}
```

```
.transaction-date {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 10px;  
  font-size: 0.95rem;  
}
```

```
.transaction-date div {  
    display: flex;  
    align-items: center;  
}
```

```
.transaction-date i {  
    margin-right: 8px;  
    color: var(--primary);  
}
```

```
.transaction-id {  
    font-family: 'Consolas', monospace;  
    font-size: 1rem;  
    color: var(--text-medium);  
    letter-spacing: 1px;  
    margin-top: 5px;  
    text-align: right;  
}
```

/ Detalles */*

```
.receipt-details {  
    margin-bottom: 25px;  
}
```

```
.detail-row {  
    display: flex;  
    justify-content: space-between;  
    padding: 12px 0;  
    border-bottom: 1px solid var(--border);  
}
```

```
.detail-row:last-child {  
  border-bottom: none;  
}
```

```
.detail-label {  
  flex: 0 0 45%;  
  font-weight: 600;  
  display: flex;  
  align-items: center;  
}
```

```
.detail-label i {  
  width: 20px;  
  margin-right: 10px;  
  color: var(--primary);  
}
```

```
.detail-value {  
  flex: 0 0 55%;  
  text-align: right;  
}
```

/ Monto y saldo */*

```
.amount-section {  
  background-color: var(--secondary-light);  
  border-radius: var(--border-radius);  
  padding: 20px;  
  margin-bottom: 25px;  
  border-left: 4px solid var(--secondary);  
}
```

```
.amount-row {  
  display: flex;  
  justify-content: space-between;  
  padding: 12px 0;  
  border-bottom: 1px dashed var(--border);  
}
```

```
.amount-row:last-child {  
  border-bottom: none;  
}
```

```
.amount-label {  
  font-weight: 600;  
  display: flex;  
  align-items: center;  
}
```

```
.amount-label i {  
  margin-right: 10px;  
  color: var(--secondary);  
}
```

```
.amount-value {  
  font-family: 'Consolas', monospace;  
  font-size: 1.2rem;  
  font-weight: 700;  
}
```

```
.amount-value.result {  
  color: var(--success);  
}
```

```
}
```

```
/* Código QR / Validación */
```

```
.receipt-validation {  
    background-color: var(--gray-light);  
    border-radius: var(--border-radius);  
    padding: 20px;  
    margin-bottom: 25px;  
    text-align: center;  
}
```

```
.qr-container {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    margin-bottom: 15px;  
}
```

```
.qr-code {  
    width: 180px;  
    height: 180px;  
    background-color: white;  
    padding: 8px;  
    border: 1px solid var(--border);  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    margin: 0 auto;  
}
```

```
.qr-code img {
```



```
width: 100%;  
height: 100%;  
object-fit: contain;  
}
```

```
.hash-reference {  
  font-family: 'Consolas', monospace;  
  font-size: 0.85rem;  
  color: var(--text-medium);  
  word-break: break-all;  
  text-align: center;  
  margin-top: 10px;  
}
```

```
/* Pie del comprobante */  
.receipt-footer {  
  text-align: center;  
  border-top: 1px solid var(--border);  
  padding: 20px;  
  color: var(--text-medium);  
  background-color: var(--gray-light);  
}
```

```
.receipt-footer p {  
  margin-bottom: 5px;  
  font-size: 0.95rem;  
}
```

```
.timestamp {  
  font-size: 0.85rem;  
  margin-top: 10px;
```

```
    color: var(--text-light);  
}
```

```
/* Botones de acción */
```

```
.actions-footer {  
    display: flex;  
    justify-content: center;  
    gap: 15px;  
    padding: 20px;  
    background-color: var(--gray-light);  
    border-top: 1px solid var(--border);  
}
```

```
.btn {  
    padding: 10px 20px;  
    border-radius: var(--border-radius);  
    font-weight: 600;  
    cursor: pointer;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    text-decoration: none;  
    transition: background-color 0.2s, transform 0.2s;  
    border: none;  
    font-size: 0.95rem;  
    flex: 1;  
    max-width: 200px;  
}
```

```
.btn i {  
    margin-right: 8px;
```

```
}
```

```
.btn-primary {  
  background-color: var(--primary);  
  color: white;  
}
```

```
.btn-primary:hover {  
  background-color: #0a5510;  
  transform: translateY(-2px);  
}
```

```
.btn-secondary {  
  background-color: #6c757d;  
  color: white;  
}
```

```
.btn-secondary:hover {  
  background-color: #5a6268;  
  transform: translateY(-2px);  
}
```

```
.btn-outline {  
  background-color: transparent;  
  border: 1px solid var(--primary);  
  color: var(--primary);  
}
```

```
.btn-outline:hover {  
  background-color: var(--primary-light);  
  transform: translateY(-2px);  
}
```

```
}
```

```
.btn-info {  
    background-color: var(--info);  
    color: white;  
}
```

```
.btn-info:hover {  
    background-color: #46b8da;  
    transform: translateY(-2px);  
}
```

```
.btn-warning {  
    background-color: var(--warning);  
    color: white;  
}
```

```
.btn-warning:hover {  
    background-color: #eea236;  
    transform: translateY(-2px);  
}
```

```
/* Sello de verificación */
```

```
.verification-seal {  
    position: absolute;  
    top: 20px;  
    right: 20px;  
    width: 100px;  
    height: 100px;  
    opacity: 0.2;  
    pointer-events: none;
```

```
}
```

```
/* Información de seguridad */
```

```
.security-info {  
    background-color: var(--primary-light);  
    border-radius: var(--border-radius);  
    padding: 15px;  
    font-size: 0.85rem;  
    color: var(--text-medium);  
    margin-bottom: 20px;  
    border-left: 4px solid var(--primary);  
}
```

```
.security-info p {  
    display: flex;  
    align-items: center;  
}
```

```
.security-info i {  
    margin-right: 10px;  
    color: var(--primary);  
}
```

```
/* Media compartir */
```

```
.share-buttons {  
    display: flex;  
    justify-content: center;  
    gap: 10px;  
    margin-bottom: 20px;  
}
```

```
.share-button {  
    width: 40px;  
    height: 40px;  
    border-radius: 50%;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    color: white;  
    font-size: 1.2rem;  
    cursor: pointer;  
    transition: transform 0.2s;  
}
```

```
.share-button:hover {  
    transform: scale(1.1);  
}
```

```
.share-whatsapp {  
    background-color: #25D366;  
}
```

```
.share-email {  
    background-color: #D44638;  
}
```

```
.share-telegram {  
    background-color: #0088cc;  
}
```

```
/* Notificación de éxito */
```

```
.success-notification {
```

```
position: fixed;
top: 20px;
right: 20px;
background-color: var(--success);
color: white;
padding: 15px 20px;
border-radius: var(--border-radius);
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
display: flex;
align-items: center;
z-index: 1000;
transform: translateY(-100px);
opacity: 0;
transition: transform 0.3s, opacity 0.3s;
}
```

```
.success-notification.show {
  transform: translateY(0);
  opacity: 1;
}
```

```
.success-notification i {
  margin-right: 10px;
  font-size: 1.2rem;
}
```

```
/* Estilos para impresión */
@media print {
  body {
    background-color: white;
    font-size: 12pt;
  }
}
```

```
}
```

```
.container {  
  height: auto;  
  padding: 0;  
}
```

```
.receipt-container {  
  box-shadow: none;  
  border: 1px solid #ddd;  
}
```

```
.actions-footer, .btn, .share-buttons, .success-notification {  
  display: none !important;  
}
```

```
.receipt-header {  
  background-color: #f0f0f0 !important;  
  color: black !important;  
  border-bottom: 1px solid #ddd;  
}
```

```
.header-transaction-info {  
  background-color: transparent !important;  
}
```

```
.header-label, .header-value {  
  color: black !important;  
}
```

```
.verification-seal {
```



```
    opacity: 0.1;
}
```

```
.transaction-type {
    background-color: #f0f0f0 !important;
    color: black !important;
    border: 1px solid #ddd;
}
```

```
.security-info {
    display: none;
}
}
```

```
/* Responsive */
@media (max-width: 768px) {
    .container {
        padding: 10px;
    }
}
```

```
.receipt-body {
    padding: 20px;
}
```

```
.receipt-title {
    font-size: 1.4rem;
}
```

```
.bank-name {
    font-size: 1.6rem;
}
```

```
.detail-row, .amount-row {  
    flex-direction: column;  
}
```

```
.detail-label, .detail-value,  
.amount-label, .amount-value {  
    width: 100%;  
    flex: 0 0 100%;  
    text-align: left;  
}
```

```
.detail-value, .amount-value {  
    margin-top: 5px;  
    padding-left: 30px;  
}
```

```
.actions-footer {  
    flex-wrap: wrap;  
}
```

```
.btn {  
    flex: 0 0 calc(50% - 10px);  
    max-width: none;  
}
```

```
.header-transaction-row {  
    flex-direction: column;  
}
```

```
.header-transaction-item {
```

```

        text-align: center;

        padding: 3px 0;
    }

    .qr-code {
        width: 150px;
        height: 150px;
    }
}

</style>
</head>
<body>
    <div class="container">
        <div class="receipt-container">
            <!-- Marca de agua -->
            <div class="watermark">COMPROBANTE</div>

            <!-- Encabezado del comprobante -->
            <div class="receipt-header">
                <div class="bank-logo">
                    
                    <div class="bank-name">Banco Mercantil</div>
                </div>
                <h1 class="receipt-title">COMPROBANTE DE TRANSACCIÓN</h1>
                <!-- Información de transacción en el encabezado -->
                <div class="header-transaction-info">
                    <div class="header-transaction-row">
                        <div class="header-transaction-item">
                            <span class="header-label">Fecha: </span>
                            <span class="header-value"><?php echo date('d/m/Y', strtotime($this->model->fecha)); ?></span>

```

```

</div>

<div class="header-transaction-item">

    <span class="header-label">Hora: </span>

    <span class="header-value"><?php echo date('H:i:s', strtotime($this->model-
>hora)); ?></span>

</div>

<div class="header-transaction-item">

    <span class="header-label">ID: </span>

    <span class="header-value"><?php echo $this->model-
>idTransaccion; ?></span>

</div>

</div>

</div>

<!-- Subtítulo dinámico según el tipo de transacción -->

<?php
$subtitleClass = "";
switch($this->model->tipoTransaccion) {
    case 1:
        $receiptType = $lang['withdrawal_receipt'];
        $transactionClass = 'withdrawal';
        $transactionName = $lang['withdrawal'];
        break;
    case 2:
        $receiptType = $lang['deposit_receipt'];
        $transactionClass = 'deposit';
        $transactionName = $lang['deposit'];
        break;
    case 3:
        $receiptType = $lang['transfer_received_receipt'];
        $transactionClass = 'transfer-in';
        $transactionName = $lang['transfer_received'];
        break;
}

```

case 4:

```
$receiptType = $lang['transfer_sent_receipt'];
```

```
$transactionClass = 'transfer-out';
```

```
$transactionName = $lang['transfer_sent'];
```

```
break;
```

default:

```
$receiptType = $lang['transaction_receipt'];
```

```
$transactionClass = '';
```

```
$transactionName = $lang['other'];
```

```
}
```

```
?>
```

```
<p class="receipt-subtitle"><?php echo $receiptType; ?></p>
```

```
</div>
```

```
<!-- Cuerpo del comprobante -->
```

```
<div class="receipt-body">
```

```
<!-- Información de seguridad -->
```

```
<div class="security-info">
```

```
<p>
```

```
<i class="fas fa-shield-alt"></i>
```

```
<?php echo $lang['receipt_security_message'] ?? 'Este documento es un  
comprobante oficial de Banco Mercantil. Verifique los datos antes de finalizar.'; ?>
```

```
</p>
```

```
</div>
```

```
<!-- Detalles de la transacción -->
```

```
<div class="receipt-details">
```

```
<div class="detail-row">
```

```
<div class="detail-label">
```

```
<i class="fas fa-university"></i>
```

```

        <span><?php echo $lang['account_number']; ?></span>

    </div>

    <div class="detail-value"><?php echo $cuenta->nroCuenta; ?></div>
</div>

<div class="detail-row">
    <div class="detail-label">
        <i class="fas fa-user"></i>

        <span><?php echo $lang['client']; ?></span>

    </div>

    <div class="detail-value"><?php echo $cliente->nombre . ' ' . $cliente-
>apellidoPaterno . ' ' . $cliente->apellidoMaterno; ?></div>

</div>

<div class="detail-row">
    <div class="detail-label">
        <i class="fas fa-exchange-alt"></i>

        <span><?php echo $lang['transaction_type']; ?></span>

    </div>

    <div class="detail-value"><?php echo $transactionName; ?></div>

</div>

<?php if (!empty($this->model->cuentaOrigen)): ?>
    <div class="detail-row">
        <div class="detail-label">
            <i class="fas fa-sign-out-alt"></i>

            <span><?php echo $lang['source_account']; ?></span>

        </div>

        <div class="detail-value"><?php echo $this->model->cuentaOrigen; ?></div>

    </div>

<?php endif; ?>

```

```

<?php if (!empty($this->model->cuentaDestino)): ?>
    <div class="detail-row">
        <div class="detail-label">
            <i class="fas fa-sign-in-alt"></i>
            <span><?php echo $lang['destination_account']; ?></span>
        </div>
        <div class="detail-value"><?php echo $this->model->cuentaDestino; ?></div>
    </div>
<?php endif; ?>

```

```

<div class="detail-row">
    <div class="detail-label">
        <i class="fas fa-align-left"></i>
        <span><?php echo $lang['description']; ?></span>
    </div>
    <div class="detail-value"><?php echo $this->model->descripcion; ?></div>
</div>

```

<!-- Información adicional - Sucursal -->

```

<div class="detail-row">
    <div class="detail-label">
        <i class="fas fa-map-marker-alt"></i>
        <span><?php echo $lang['branch'] ?? 'Sucursal'; ?></span>
    </div>

```

```

    <div class="detail-value"><?php echo $cuenta->sucursal ?? 'Casa
Matriz'; ?></div>
</div>

```

<!-- Cajero/Atendido por -->

```

<div class="detail-row">

```

```

        <div class="detail-label">

            <i class="fas fa-user-tie"></i>

            <span><?php echo $lang['attended_by'] ?? 'Atendido por'; ?></span>

        </div>

        <div class="detail-value"><?php echo $this->model->usuario ??
$lang['online_transaction'] ?? 'Transacción en línea'; ?></div>

    </div>

</div>

```

<!-- Sección de montos -->

```

<div class="amount-section">

    <div class="amount-row">

        <div class="amount-label">

            <i class="fas fa-money-bill-wave"></i>

            <span><?php echo $lang['amount']; ?></span>

        </div>

        <div class="amount-value">

            <?php

            if ($cuenta->tipoMoneda == 1) {

                echo 'Bs. ' . number_format($this->model->monto, 2);

            } else {

                echo '$ ' . number_format($this->model->monto, 2);

            }

            ?>

        </div>

    </div>

</div>

```

```

<div class="amount-row">

    <div class="amount-label">

        <i class="fas fa-wallet"></i>

        <span><?php echo $lang['resulting_balance']; ?></span>

    </div>

</div>

```


</div>

<div class="amount-value result">

<?php

if (\$cuenta->tipoMoneda == 1) {

echo 'Bs. ' . number_format(\$this->model->saldoResultante, 2);

} else {

echo '\$ ' . number_format(\$this->model->saldoResultante, 2);

}

?>

</div>

</div>

</div>

<!-- Validación con QR corregido -->

<div class="receipt-validation">

<div class="qr-container">

<div class="qr-code">

</div>

</div>

<p><?php echo \$lang['scan_qr_code'] ?? 'Escanee el código QR para verificar la autenticidad'; ?></p>

<div class="hash-reference">

<small><?php echo \$lang['transaction_reference'] ?? 'Referencia'; ?>: <?php echo \$this->model->hash; ?></small>

</div>

</div>

<!-- Pie del comprobante -->

<div class="receipt-footer">

<p><?php echo \$lang['receipt_thank_you'] ?? '¡Gracias por confiar en Banco Mercantil!'; ?></p>

<p><?php echo \$lang['receipt_valid_without_signature'] ?? 'Este comprobante es válido sin firma ni sello.'; ?></p>

<p><?php echo \$lang['receipt_contact_info'] ?? 'Para cualquier consulta, comuníquese al 0800-BANCO (22626)'; ?></p>

<p class="timestamp"><?php echo \$lang['generated_on'] ?? 'Generado el'; ?><?php echo date('d/m/Y H:i:s'); ?></p>

</div>

</div>

<!-- Botones de acción -->

<div class="actions-footer">

<button type="button" class="btn btn-primary" onclick="window.print();">

<i class="fas fa-print"></i> <?php echo \$lang['print']; ?>

</button>

<a href="index.php?controller=transaccion&action=ver&id=<?php echo \$this->model->idTransaccion; ?>" class="btn btn-secondary">

<i class="fas fa-arrow-left"></i> <?php echo \$lang['back']; ?>

<button type="button" class="btn btn-info" onclick="downloadPDF()">

<i class="fas fa-file-pdf"></i> <?php echo \$lang['download_pdf'] ?? 'Descargar PDF'; ?>

</button>

<button type="button" class="btn btn-outline" onclick="sendByEmail()">

<i class="fas fa-envelope"></i> <?php echo \$lang['send_by_email'] ?? 'Enviar por Email'; ?>

</button>

<button type="button" class="btn btn-warning" onclick="reportIssue()">

```
        <i class="fas fa-exclamation-triangle"></i> <?php echo $lang['report_issue'] ??  
'Reportar'; ?>
```

```
    </button>
```

```
</div>
```

```
<!-- Sello de verificación -->
```

```

```

```
</div>
```

```
</div>
```

```
<!-- Notificación de éxito (oculta por defecto) -->
```

```
<div id="successNotification" class="success-notification">
```

```
    <i class="fas fa-check-circle"></i>
```

```
    <span id="notificationMessage"></span>
```

```
</div>
```

```
<script>
```

```
    // Función para mostrar notificación
```

```
    function showNotification(message) {
```

```
        const notification = document.getElementById('successNotification');
```

```
        const messageElement = document.getElementById('notificationMessage');
```

```
        messageElement.textContent = message;
```

```
        notification.classList.add('show');
```

```
        setTimeout(() => {
```

```
            notification.classList.remove('show');
```

```
        }, 3000);
```

```
    }
```

```
    // Función para descargar PDF
```

```
    function downloadPDF() {
```

```

// Aquí se implementaría la lógica real de descarga
showNotification('El PDF ha sido descargado correctamente');
}

// Función para enviar por email
function sendByEmail() {
    // Simulación de envío de email
    setTimeout(() => {
        showNotification('El comprobante ha sido enviado por email');
    }, 1000);
}

// Función para reportar un problema
function reportIssue() {
    // Aquí se implementaría el formulario o la lógica para reportar
    const reason = prompt('Por favor, describa el problema con este comprobante:');
    if (reason) {
        showNotification('Su reporte ha sido enviado. Gracias por su feedback.');
```

```
const body = 'Adjunto encontrará su comprobante de transacción #<?php echo $this->model->idTransaccion; ?> del Banco Mercantil.';

window.location.href =
`mailto:?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`;

showNotification('Abriendo su cliente de correo...');

}

function shareViaTelegram() {

const message = 'Comprobante de transacción Banco Mercantil #<?php echo $this->model->idTransaccion; ?>';

window.open(`https://t.me/share/url?url=${encodeURIComponent(window.location.href)}
&text=${encodeURIComponent(message)}`, '_blank');

showNotification('Compartiendo por Telegram...');

}

</script>

</body>

</html>
```